Mapping (Version 1.0)

`Crypto_PAKEValues_Responder := (w0, L)` where `w0` and `L` SHALL be computed as follows:

```
byte w0s[CRYPTO_W_SIZE_BYTES] || byte w1s[CRYPTO_W_SIZE_BYTES] =
     (byte[2 * CRYPTO_W_SIZE_BYTES])
     bit[2 * CRYPTO_W_SIZE_BITS]
     Crypto_PBKDF(passcode, salt, iterations, 2 * CRYPTO_W_SIZE_BITS)
byte w0[CRYPTO_GROUP_SIZE_BYTES] = w0s mod p
byte w1[CRYPTO_GROUP_SIZE_BYTES] = w1s mod p
byte L[CRYPTO_PUBLIC_KEY_SIZE_BYTES] = w1 * P
```

where:

- `passcode`, is the Passcode defined in Section 5.1.1.6, “Passcode”.
- `p` is the order of the elliptic curve to be used.
- Both `w0s` and `w1s` SHALL have a length equal to `(CRYPTO_GROUP_SIZE_BYTES + 8)`.
- `salt` and `iterations` are extracted from the `Crypto_PBKDFParameterSet` values.
- `P` is the generator of the underlying elliptic curve.
- When the computation of `Crypto_PAKEValues_Responder` is done, fields `w0` and `L` SHALL be stored in the Responder and `w1` SHALL NOT be stored in the Responder.
- The pair `(w0,L)` is also referred to as Commissionee PAKE input or verification value

3.10.1. Computation of pA

Mapping (Version 1.0)

```
Crypto_pA(Crypto_PAKEValues_Initiator) :=
     byte pA[CRYPTO_PUBLIC_KEY_SIZE_BYTES]
```

`pA` is in uncompressed public key format as specified in section 2.3 of SEC 1. `pA` SHALL be computed as specified in SPAKE2+.

3.10.2. Computation of pB

Mapping (Version 1.0)

```
Crypto_pB(Crypto_PAKEValues_Responder) :=
     byte pB[CRYPTO_PUBLIC_KEY_SIZE_BYTES]
```

`pB` is in uncompressed public key format as specified in section 2.3 of SEC 1. `pB` SHALL be computed as specified in SPAKE2+.